

Tracking LLVM Performance Over Time

Feasibility Study

Wenhao Li

2026.04.16

1 Introduction

In the current field of high-performance computing (HPC), compilers serve as the fundamental bridge between high-level programming languages and the low-level operations executed by hardware. They provide the essential infrastructure for performance-critical tasks, such as instruction scheduling, register allocation, and hardware-specific code generation. Within the compiler ecosystem, LLVM [1] has emerged as a predominant framework for modern compiler construction, underlying widely used toolchains for languages such as C/C++ [2], Rust [3], and Swift [4]. This widespread adoption is attributed to its modular architecture, its support for a diverse range of hardware targets, and its extensive suite of optimization passes.

The LLVM framework is under constant development, with frequent changes that present significant engineering challenges. The project maintains a rapid release schedule; over the past year alone, there have been two major version cycles and approximately 15 canonical releases. Because optimization algorithms evolve with each of these versions, these shifts can lead to unpredictable fluctuations in both execution speed and memory usage for scientific applications—sometimes even resulting in regressions. Despite the potential for such shifts, the industry still lacks a robust, reliable tool to track and record these comprehensive performance trends over the long term.

To solve these problems, this project plans to design and build a performance testing system that is easy to use and highly automated. It is specially made to compare and check how performance changes across different LLVM versions. The system will use the Lit [5] tool inside LLVM to run the tests. It will focus on two very typical test suites: the official LLVM test suite and the RAJA performance test suite [6], which is for C++ and supports various parallel programming models such as OpenMP [7] and CUDA [36]. By building a system that automatically downloads source code, finishes the compilation, runs the tests, analyses the results, and turns them into visualisations, this project can provide a serious and useful tool for researchers and developers when they need to evaluate and pick a compiler version.

2 Background and Literature Review

In the current field of high-performance computing, the LLVM compiler framework is used very widely [8, 9, 10]. It uses a very clear “three-stage modular design,” which means the whole compilation process is divided into frontend parsing, middle-end optimization, and backend machine code generation. This design makes the system very easy to expand. However, as the compilation process gets more complex, it becomes harder to accurately track its performance [11, 12, 13]. The LLVM project follows a very regular release schedule, putting out a major version about every six months. Because compiler optimization basically uses a set of rules to find a relatively good solution in a very large search space, any change to the compiler’s internal structure or logic can cause the running speed and memory usage of scientific applications to show clear and irregular fluctuations. Some actual studies have proven that version updates do not always mean the execution efficiency gets better. Some deep optimizations made for specific code patterns might accidentally cause performance to drop for other types of tasks. Therefore, it is hard to accurately evaluate a compiler’s actual performance just by using static analysis methods. Comparing running performance across versions in a real hardware environment is necessary.

To monitor how compiler performance changes over time, the industry and the development community have released many testing tools and frameworks. However, if we want to quickly

compare and analyze different compiler versions on a single node, these existing solutions often show some limitations:

- Lit is a lightweight testing tool that comes with the LLVM system. It is mainly used to run a large number of regression tests at the same time [5]. However, its design is mainly as a test execution engine. It did not consider the need for tracking performance over a long period. It cannot automatically collect and summarize test data, and it does not have a special storage mechanism to record and compare performance changes across historical versions.
- The LLVM community developed the LNT system specifically to monitor compilation time and execution efficiency [14]. But the architecture of this system is quite complex, and the public test servers often run unstably. Also, it is not very good at integrating new test requests. Most importantly, the user experience is poor, so developers in the community are not very willing to use it. It is hard to meet the needs for quick verification [15].
- HPC Testing frameworks like ReFrame [16] or JUBE [17]. ReFrame provides a Python-based framework to automate the build, execution, and sanity checking of benchmarks across diverse system partitions, while JUBE utilizes XML-based templates to manage complex parameter studies and systematically extract performance data from log files. While frameworks like ReFrame and JUBE offer robust features for large-scale systems, their steep learning curve and complex XML/Python abstractions can be prohibitive for individual researchers [18, 19]. For localized tasks such as LLVM performance evaluation on a single workstation, the system overhead and deployment complexity of these "heavy" frameworks often outweigh their benefits [20]. Since this project evaluates LLVM compiler performance on a single machine, using these heavy frameworks adds unnecessary system overhead and makes deployment harder.

Based on the research and analysis above, this project did not choose those overly complex benchmarking systems. Instead, it uses Python scripts to achieve automated performance tracking. It wraps Lit to run the LLVM test suite, while using native execution programs for the RAJA suite. To ensure the test results are representative under limited resource conditions, the system integrates two very core test suites:

- **Official LLVM Test Suite:** This is a basic tool for evaluating code generation quality. It contains many representative open-source applications and test libraries specifically for evaluating vectorization performance [21]. This suite is naturally compatible with the Lit tool and supports a special testing mode, ensuring the data collected is consistent with the optimization goals of the official LLVM community, which primarily focus on maximizing execution performance, minimizing code size, and maintaining compilation efficiency [22].
- **RAJA Performance Test Suite:** Suggested by the project proposer, Michele Weiland (EPCC), this suite is utilized because many modern high-performance computing apps rely heavily on C++ template programming to build cross-platform abstraction layers [23]. This really tests the compiler's ability to handle code inlining and remove redundant code. Traditional general tests often fail to find defects in the compiler when it handles these advanced features [24]. The RAJA suite is specifically used to measure the running efficiency of loop calculation kernels [6]. A loop calculation kernel is a computationally intensive code segment, typically a loop, that performs the core numerical operations of an algorithm and often dominates the overall execution time. It calculates a metric called "Figure of Merit" (FOM), which compares the running time of raw code versus abstract code to measure the compiler's optimization level [24]. If the level drops too much, it means the compiler has a clear performance regression when handling modern C++ code [24]. These problematic code snippets can be given directly to compiler developers

for later fixing work [24].

Although this project currently focuses on the two test suites mentioned above, a complete performance evaluation system needs to consider more dimensions. For example, OpenMP-related benchmarks can be used to evaluate the overhead of the runtime library during thread synchronization [7]. Additionally, the SOLLVE OMPVV test suite can be used to check if the compiler's meaning stays correct when implementing new programming standards [25].

3 Preliminary Investigations and Findings

This section explains the exploratory work and proof of concept done during the project's preparation phase. So far, a complete automated process has been successfully built. This process covers everything from getting the source code and compiling it automatically to running the benchmark tests, analyses the results and showing the data in visualisations.

3.1 Developing an Automated Framework

The development of this project was done in a step-by-step way. In the beginning, LLVM source code was configured, compiled, and tested by typing commands manually. This was done to get a better understanding of how the underlying build infrastructure and its mechanics work. After getting used to how the workflow works, these steps were put into basic Bash scripts so the testing process could be run by executing a single command.

To make the system easier to maintain and expand, and to make sure the automation parts work well with the data analysis and visualization code later, the automation framework rewritten into Python. During this process, YAML [26] configuration files were added. This separates the logic from the specific settings (like LLVM version numbers or build parameters), making the system much more flexible [27].

3.2 Environment Control and Resource Optimization

Early compilation work was conducted in a local Ubuntu 24.04.3 LTS environment, and the testing process was later moved to the EIDF VM, which runs Ubuntu 24.04.4 LTS.

During the process of compiling LLVM from source code, the system ran out of memory and triggered a kernel error. At first, the EIDF virtual machine (4-core CPU and 4GB RAM) tried to avoid this by limiting the number of parallel tasks, but this made a single compilation take 3838 seconds. Later, after upgrading the VM to a 16-core CPU and 32GB RAM, the compilation time dropped to 1126 seconds, which made the testing much faster.

Also, to stop old CMake cache from making the compilation fail, a strict cleaning step was added to the script. This forces the system to clean out old folders before every build. At the same time, the system uses separate log files for different information, separating the progress messages on the screen from the long compilation logs. This makes sure the process is easy to see and can be checked later.

3.3 Core Technical Challenges and Solutions

When setting up the RAJA performance suite to check the OpenMP parallel model, there was a linking error where the Clang [28] compiler—which serves as the C-family front-end for the LLVM project—could not find the custom OpenMP library. To fix this, the pipeline was modified to avoid relying on system-level environment variables for library resolution, which are unreliable in this context. Instead, it embeds dynamic runtime search paths (using the RPATH parameter, which embeds specific library search paths directly into the executable) into the linker flags. This forces the benchmark to connect to the newly compiled LLVM library when the program runs.

While running loop vectorization tests in the LLVM test suite, the system caught an unusual floating-point error at runtime. By using the `git bisect` tool [37] to automatically search through LLVM test suite code updates—as no issue had yet been reported on the LLVM GitHub repository for the bug—it was found that the crash was caused by a bug recently added by the main developers. This problem was fixed in a later patch (commit 488a96b). This process proved that this system is really able to find and locate problems in the underlying code generation.

3.4 Data Parsing and Visualization Results

The official test suite mainly gives data in JSON [29] format, while the RAJA suite gives data in comma-separated values (CSV) format. To handle all this data, the system has a standard data extraction module. This module cleans the data from different sources and formats and turns it into a common internal structure—a unified record that captures metadata (such as suite and compiler versions, run IDs, and test status) and core performance metrics (including execution/compile times, binary sizes, bandwidth, and FLOPS)—which is a good foundation for later statistical analysis.

Currently, the work has covered several versions from LLVM 17.0.0 to 22.1.0. Based on the cleaned data, the system uses the Plotly [30] library to make interactive web reports [31], as shown in Figure 1.

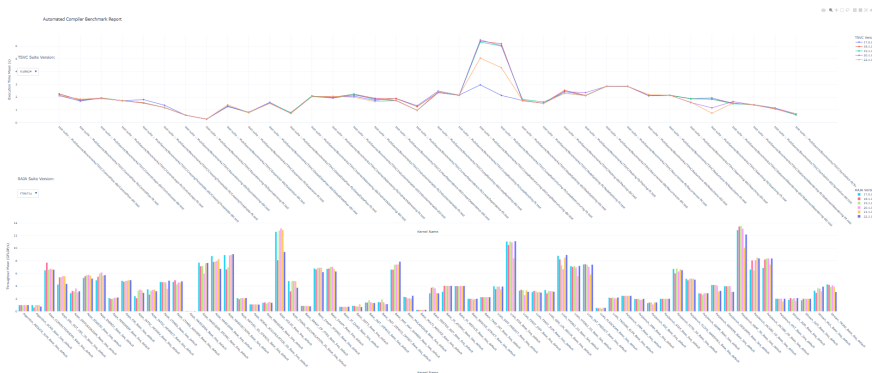


Figure 1: Screenshot of the interactive web report generated by the system.

For the charts, line charts are currently used to show the execution time of vectorization tests,

but there are plans to change these to bar charts to avoid confusing users. This confusion arises because the current graphs display too much information at once; switching to bar charts will allow users to better control and filter what is presented. Also, bar charts are used to clearly show how much work the different LLVM versions can do when handling RAJA kernels.

4 Project Proposal

4.1 Core Research Questions and Hypotheses

In the field of high-performance computing, the optimization processes of modern compilers are very complicated. This makes the performance of a compiler often change in irregular ways when it is updated to a new version. This project aims to build and improve a testing framework that is highly automated and can track LLVM compiler performance across different versions.

The core hypothesis of this project is: by building automated workflows, the system can compile specific versions of LLVM whenever needed and run different benchmark suites (like the official LLVM test suite and the RAJA performance suite). By cleaning and organizing the different types of output data, the system can measure and show where the compiler's performance got better or worse as the architecture changed. This will give high-performance computing developers solid, data-driven facts to help them choose which compiler version to use.

4.2 Minimum Viable Product

To make sure the graduation project is finished successfully, the final system must have at least the following core modules. These modules are the basic requirements for the project:

- **Full-process automated pipeline:** Provide a stable automation engine implemented in Python. This engine should use a YAML configuration file to specify a full loop of "getting source code, building in an isolated environment, running tests, and saving logs" without needing any manual help.
- **Strict error handling and recovery:** The system must be robust enough to handle errors. If a certain compiler version fails to build or crashes while running, the system should not just quit without a message. It must record the error accurately, safely skip that specific task, and keep going with the rest of the testing queue.
- **Data parsing adapters for different formats:** Build a layer to extract data that can take JSON data and CSV data from specific suites. It should clean this data and turn it into a common internal format to make it easy to process later.
- **Static benchmark report generator:** Using the saved test data, the system should automatically create static HTML (with embedded JavaScript) charts. These charts will clearly show core metrics like execution time and memory use for different test cases across different compiler versions.

4.3 Extended Goals and Future Work

If the basic goals are finished, the project may expand further. The items below fall into two categories. Some — such as advanced visualisation, performance change detection, automated

version monitoring, and test subset selection — are scheduled as optional tasks in the workplan (Section 5) and will be attempted if time permits. The remaining items, including allowing users to upload custom test sets, adding new HPC benchmark suites, and statistical significance testing, are preliminary thoughts for future work beyond the current project scope and have no corresponding workplan entries.

Advanced Data Interaction and Display

- **Simple web application:** Turn the data processing scripts into a web app using Python, using frameworks like Streamlit [32] or Dash [33]—which are open-source tools that allow for the rapid creation of data-driven interfaces directly in Python. This avoids difficult front-end coding and focuses on showing data in an interactive way.
- **Interactive kernel filtering and better visuals:** Add ways to filter results so users can look at the performance of just one calculation kernel. Also, use different patterns or shading in charts to help users tell different LLVM versions apart more easily.

Giving the user more control

- **Test subset selection:** Create a feature that lets users choose exactly which tests to run. This is meant to solve problems where output files are too large or tests take too long, which reduces the work the computer has to do and makes the final report easier to read.
- **Letting users add their own tests:** Allow users to upload their own code sets. The system will then automatically create the configuration files and include that code in the standard testing process.

Making the framework more pluggable

- **Support for multiple test suite versions and parser formats:** Make the data parser more flexible so it can handle changes in data fields from different versions of a test suite and support different parser formats. Or, set up strict version control for all dependencies.
- **Adding new HPC evaluation frameworks:** Besides the two current test suites, look into adding OpenMP micro-benchmarks to measure the extra time used for thread management. Or, add the SOLLVE OMPVV test suite to check if the compiler follows programming rules correctly.

Smart Analysis and Alerts

- **Automatic performance change detection:** After comparing versions, the system should automatically calculate the differences in performance. It should list the kernels where performance changed the most (either improved or regressed), filtered by a user-defined significance threshold so that only meaningful changes are highlighted.
- **Statistical significance testing:** Allow the system to run the same test several times and use basic statistics (like a t-test). The system will use this to decide if a small change in speed is really caused by a change in the compiler or if it is just normal noise from the computer system.
- **Automated LLVM version monitoring:** Set up a CRON job or GitLab CI [38] pipeline that periodically checks the LLVM repository for new tags or commits. When a change is detected, it automatically triggers the testing framework, enabling continuous and hands-free performance tracking without manual intervention.

5 Workplan

To make sure the graduation project finishes on time by the August 14, 2026 deadline, this plan splits the work into several tasks. The whole work is divided into four main parts: infrastructure refactoring, data collection, advanced visualisation, and thesis writing.

The official development work is planned to start in mid-to-late April after the proposal is submitted. To deal with possible technical problems or delays accessing the EIDF VM, this plan includes enough buffer time between key dates.

Table 1 is the detailed list of tasks and the schedule until the end of the project:

5.1 Project Task and Schedule Table

Table 1: Project Task and Schedule

Task Category	Task Description	Estimated Time	Start Date	End Date
Task 1: Infrastructure Refactoring	1.1 Workflow engine evaluation: Investigate whether Snakemake or Nextflow would simplify error handling and logging compared to the current Python pipeline. Assess the learning curve and estimate the cost of migrating existing work to such a framework. Make a go/no-go decision before committing to the automation architecture.	3 days	2026-04-20	2026-04-22
	1.2 Error handling for automation scripts: Add stable error-handling logic to the Python pipeline to ensure that if a compiler version fails to build, the system catches the error, skips that task, and continues with the rest of the queue.	1 week	2026-04-23	2026-04-29
	1.3 Status recovery and log saving: Improve the pipeline so it can resume from where it stopped after an interruption, and improve log organisation for long-term storage.	1 week	2026-04-30	2026-05-06
	1.4 Data format adaptation for different versions: Rewrite and upgrade the data extraction layer to handle changes in output formats across test suite versions, producing a single unified internal format. Moving this task into the infrastructure phase ensures the pipeline is designed to be extensible from the outset.	1 week	2026-05-07	2026-05-13

(Continued on next page)

(Continued)

Task Category	Task Description	Estimated Time	Start Date	End Date
	1.5 Custom test subset settings: Update the configuration logic to let users select exactly which tests to run. Completing this early makes it immediately useful during data collection and reduces unnecessary computation during iterative development.	1.5 weeks	2026-05-14	2026-05-24
	1.6 Interactive views and better visuals: Build richer web-based reports using Plotly. Add kernel-level filters and visual aids to distinguish compiler versions. For large volumes of results, consider a lightweight web server solution (e.g. Streamlit or Dash) in place of static HTML and JavaScript. This task is included in the infrastructure phase as it directly relates to the <i>Static benchmark report generator</i> component of the MVP.	2 weeks	2026-05-25	2026-06-07
Task 2: Data Collection	2.1 Large-scale data collection: Use the EIDF VM to automatically compile and run LLVM versions 18.x through 20.x to collect performance data for the final analysis. Also run the automated test framework on ARCHER2 as a real HPC resource to validate portability across platforms.	1.5 weeks	2026-06-08	2026-06-18
	2.2 Automated LLVM version monitoring (Optional): Set up a CRON job or GitLab CI pipeline that periodically monitors the LLVM repository for new tags or commits and automatically triggers the testing framework when a change is detected.	0.5 weeks	2026-06-19	2026-06-22
Task 3: Performance Analysis	3.1 Automatic performance change detection engine: Develop an algorithm that automatically compares versions and identifies kernels with the most significant performance changes (both improvements and regressions). Support user-defined thresholds so that only changes exceeding a specified bound are highlighted.	1.5 weeks	2026-06-23	2026-07-03
Task 4: Thesis Writing and Final Review	4.1 Drafting the outline and core content: Write the introduction, background and literature review, research methods, and the design of the automation system.	3 weeks	2026-07-04	2026-07-24

(Continued on next page)

(Continued)

Task Category	Task Description	Estimated Time	Start Date	End Date
	4.2 Deep analysis and evaluation of results: Analyse the performance data from the EIDF VM and ARCHER2, and write the analysis chapter. Use statistics to show how compiler updates actually affect different tasks.	2 weeks	2026-07-25	2026-08-07
	4.3 Final review and finishing deliverables: Proofread the whole paper. Clean up the code repository and finish the README and running guides. Use the final buffer period for any last-minute problems.	1 week	2026-08-08	2026-08-14
Total	~17 weeks across 4 modules, 12 tasks (Task 2.2 is optional)		2026-04-20	2026-08-14

6 Resources Estimation

The development and automated testing work of this project depend heavily on computing resources and storage space. Based on earlier tests and data measurements on the EIDF VM, here are the resource estimates, how they are currently used, and plans for when resources are limited.

6.1 Computing Resource Needs

Table 2 summarizes the computing resources used in this project and the purpose of each platform.

Table 2: Computing Resource Specifications

Platform	Configuration	Purpose
Local Machine	4-core CPU, 4 GB RAM	Initial development and exploratory compilation work.
EIDF VM	16-core CPU, 32 GB RAM	Primary platform for development, testing, and large-scale data collection.
ARCHER2	~16 cores	Validation of framework portability on an actual HPC system.

6.2 Storage Resource Needs

Detailed storage consumption for the compiler versions, test suites, and projected total capacity requirements are provided in Table 3.

Table 3: Storage Space Usage and Estimation

Category	Current Usage / Details	Capacity Specification
Compiler	LLVM source code, temporary builds, and versions 17.0.0 to 22.1.0.	~24 GB total (2.3–2.7 GB per installed version).
Test Suites	LLVM official suite and RAJA performance suite.	LLVM: 2.5 GB (source) / 277 MB (build). RAJA: 266 MB (source) / 24 MB (build).
Data & Logs	Experimental results in JSON and CSV formats.	~5 MB per version; approximately 75 MB for 15 versions.
Total Current Final Estimate	Main testing folder on EIDF. Estimated for 15 selected versions: 15 installed compilers $\times$ 2.5 GB + 2.8 GB shared test suites + negligible results.	~27 GB. ~41 GB.

7 Data Access and Ethics

All tools and test suites used in this project (like LLVM source code and the RAJA performance suite) are public open-source software. There is no need to sign non-disclosure agreements or ask for special data access.

This research only involves counting and analyzing the running time of computer programs. It does not involve any human participants or personal sensitive data, so no formal ethical approval is required.

8 Risk Analysis

During this project, there might be potential risks in areas like technical implementation, resource scheduling, external dependencies, and data management. To make sure the graduation project is delivered on time and with good quality, this section evaluates the core risks using a qualitative method and sets up plans to handle them.

The risk evaluation is based on two parts: the chance of it happening (unlikely, possible, very likely, and certain) and the level of impact (negligible, medium, serious, and extreme).

As shown in Figure 2, these two dimensions together determine an overall *importance* level for each risk, ranging from *trivial* and *low* through *medium* and *high* to *critical*. The importance assigned to each risk identified below is derived from this matrix.

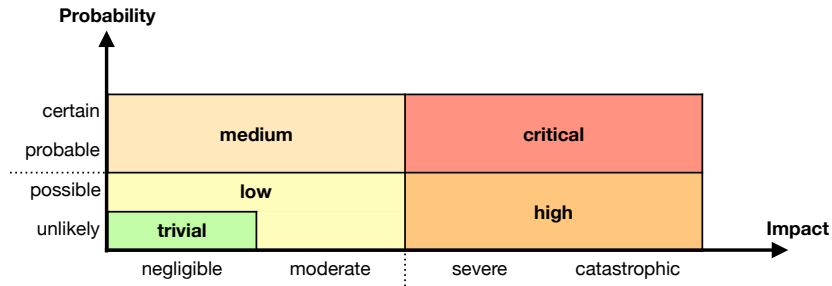


Figure 2: Risk importance matrix: probability vs. impact.

Table 4 summarises the identified risks, their evaluations, and the corresponding mitigation strategies.

Table 4: Risk Assessment Summary

Risk	Description	Prob.	Impact	Import.	Mitigation Strategy
Limited VM Resources, Insufficient Memory, or Storage Overflow	The EIDF VM or ARCHER2 may be unavailable due to maintenance or queue delays. Compiling LLVM requires substantial RAM; insufficient memory prevents builds entirely. Storing many compiler versions may exceed storage limits.	Possible	Serious. Insufficient memory halts the entire pipeline; unavailability or a full disk delays data collection.	High	(1) If memory is insufficient, request a new EIDF VM with more resources. (2) Delete temporary build folders and old installations; keep only small result files. (3) Fall back to a local machine with reduced parallelism if both the EIDF VM and ARCHER2 are unavailable.
Changes in Test Output Formats	If a test suite releases a new version, its output file structure may change, causing the parser to fail and producing empty charts with no valid analysis.	Possible	Medium. No valid performance data can be produced; charts will be empty.	Low	(1) Lock all external repository versions in the YAML configuration file. (2) Extend the parser to support multiple format versions by detecting the format at runtime and dispatching to the appropriate handler. (3) Add error-handling so missing or unrecognised fields are reported without crashing.

(Continued on next page)

(Continued)

Risk	Description	Prob.	Impact	Import.	Mitigation Strategy
Hardware Failure	The local machine, EIDF VM, or ARCHER2 could suffer a hardware failure, causing potential loss of code and scripts and requiring environment re-configuration.	Unlikely	Serious. Code backed up to ECDF GitLab is safe; data files can be regenerated by rerunning the pipeline. Recovery would cause delays.	High	(1) Push code to ECDF GitLab frequently; data files are small and reproducible so code backup alone is sufficient. (2) If the EIDF VM fails, request a replacement VM; if ARCHER2 is unavailable, fall back to the EIDF VM for HPC validation—provisioning a replacement VM is a proven and straightforward process.
Personal Illness or Health Issues	As a single-developer project, extended illness would halt all development and data collection, directly threatening the August 2026 submission deadline.	Unlikely	Serious. Could compress thesis-writing time or in the worst case make on-time submission impossible.	High	(1) Follow the workplan strictly; built-in buffer time (see Section 5) tolerates short interruptions. (2) Maintain clear documentation and commit frequently to minimise re-orientation time after recovery. (3) Notify the supervisor promptly to allow early discussion of timeline or scope adjustments.

9 Outline of the Dissertation Report

Table 5: Dissertation Report Outline

Ch.	Title	Key Sections
1	Introduction	Research Background; Problem Statement; Research Goals and Hypotheses; Main Contributions
2	Background and Related Work	LLVM Architecture and Release Cycle; Limitations of Existing Testing Frameworks; Evaluation of Benchmark Suites
3	System Architecture and Implementation	Automated Pipeline Design; Data Adaptation Layer; Visualisation Approach; Engineering Challenges and Solutions
4	Experimental Setup and Execution	Running Environment; Test Samples and Scope; Data Collection Method
5	Results Analysis and Performance Evaluation	General Performance Trends; Detailed Kernel Analysis; Automatic Performance Change Detection; Statistical Significance Testing
6	Discussion	Practical Implications; Evaluation of the Framework; Research Limitations
7	Conclusion and Future Work	Research Summary; Future Research Directions

References

- [1] LLVM. *The LLVM Compiler Infrastructure* <https://llvm.org/>.
- [2] LLVM Project. *Clang: a C language family frontend for LLVM*. <https://clang.llvm.org/>.
- [3] Rust Team. *The rustc book: LLVM Backend*. <https://rustc-dev-guide.rust-lang.org/backend/updating-llvm.html>.
- [4] Swift.org. *Swift Compiler Architecture*. <https://www.swift.org/documentation/swift-compiler/>.
- [5] LLVM. *lit - LLVM Integrated Tester — LLVM 23.0.0git documentation*. <https://llvm.org/docs/CommandGuide/lit.html>.
- [6] RAJAPerf. *RAJA Performance Suite — RAJAPerf 2025.03.0 documentation*. <https://rajaperf.readthedocs.io/>.
- [7] OpenMP. *OpenMP Evaluation Resources*. <https://www.openmp.org/resources/openmp-benchmarks/>.
- [8] NVIDIA Corporation. *NVVM IR Specification: A Compiler IR for Graphics and Parallel Computing based on LLVM*. <https://docs.nvidia.com/cuda/nvvm-ir-spec/index.html>.
- [9] SC Conference. *Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*. <https://llvm-hpc-workshop.github.io/>.
- [10] Julia Documentation. *Julia's Use of LLVM for Just-In-Time (JIT) Compilation*. <https://docs.julialang.org/en/v1/devdocs/jit/>.
- [11] Chris Lattner, et al. MLIR: A Compiler Infrastructure for the End of Moore's Law, arXiv, 25 Feb 2020, arXiv:2002.11054. <https://arxiv.org/abs/2002.11054>.
- [12] Aras Pranckevičius. *Clang time-trace is coming*. <https://aras-p.info/blog/2019/01/16/time-trace-timeline-flame-chart-profiler-for-Clang/>.
- [13] Chris Lattner. *The Architecture of Open Source Applications: LLVM*. <https://aosabook.org/en/llvm.html>.
- [14] LNT. *LNT - LLVM Performance Tracking Software* <https://llvm.org/docs/Lnt/>.
- [15] nikic. *LLVM: The bad parts - nikic's Blog*. <https://www.npopov.com/2026/01/11/LLVM-The-bad-parts.html>.
- [16] ReFrame. *Enabling Continuous Testing of HPC Systems Using ReFrame* <https://reframe-hpc.readthedocs.io/en/stable/>.
- [17] JUBE. *Welcome to JUBE's documentation!* <https://apps.fz-juelich.de/jsc/jube/docu/index.html>.
- [18] V. Karakasis, et al. *A Regression Framework for Checking the Health of Large HPC Systems*. https://cug.org/proceedings/cug2017_proceedings/includes/files/pap122s2-file1.pdf.
- [19] M. Gonzalez and J. Nederveld. *Analyzing Frameworks for HPC Systems Regression Testing*. https://cdn.lanl.gov/files/23-gonzalez-nederveld_49238.pdf.
- [20] JUBE Team. *JUBE Benchmarking Environment Tutorial*. <https://apps.fz-juelich.de/jsc/jube/docu/tutorial.html>.

- [21] LLVM. *LLVM Testing Infrastructure Guide — LLVM 23.0.0git documentation*. <https://llvm.org/docs/TestingGuide.html>.
- [22] LLVM: *A compilation framework for lifelong program analysis & transformation* <https://llvm.org/pubs/2003-09-30-LifelongOptimizationTR.pdf>.
- [23] Davis et al. An Evaluative Comparison of Performance Portability across GPU Programming Models, arXiv, 14 Feb 2024. arXiv:2402.08950. <https://arxiv.org/html/2402.08950v1>.
- [24] LLNL. *RAJA Performance Suite*. https://asc.llnl.gov/sites/asc/files/2020-09/rajaperfuite_summary-2018.02.06.pdf.
- [25] Thomas Huber, Swaroop Pophale, Nolan Baker, Michael Carr, Nikhil Rao, Jaydon Reap, Kristina Holsapple, Joshua Hoke Davis, Tobias Burnus, Seyong Lee, David E. Bernholdt, Sunita Chandrasekaran, ECP SOLLVE: Validation and Verification Testsuite Status Update and Compiler Insight for OpenMP. Accepted and to be published with P3HPC @ SC22. pp. 123-135, doi: 10.1109/P3HPC56579.2022.00017.
- [26] YAML. *YAML: YAML Ain't Markup Language* <https://yaml.org/>.
- [27] config.yml <https://git.ecdf.ed.ac.uk/epcc-msc-projects/full-time/2025-26/s2778911/-/blob/main/scripts/config.yml>.
- [28] Clang. *Clang: a C language family frontend for LLVM* <https://clang.llvm.org/>.
- [29] JSON. *Introducing JSON* <https://www.json.org/json-en.html>.
- [30] Plotly. *Welcome to Plotly Documentation* <https://docs.plotly.com/>.
- [31] Interactive Benchmark Report. *Hosted on ECDF Git Repository*. https://git.ecdf.ed.ac.uk/epcc-msc-projects/full-time/2025-26/s2778911/-/blob/main/benchmark_report.html.
- [32] StreamLit. *Streamlit documentation* <https://docs.streamlit.io/>.
- [33] Dash. *Dash Documentation & User Guide | Plotly* <https://dash.plotly.com/>.
- [34] LLVM. *test-suite Guide — LLVM 23.0.0git documentation*. <https://llvm.org/docs/TestSuiteGuide.html>.
- [35] SC19. *Enabling Continuous Testing of HPC Systems using ReFrame - SC19*. https://sc19.supercomputing.org/proceedings/workshops/workshop_files/ws_hust104s1-file1.pdf.
- [36] NVIDIA Corporation. *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [37] Git. *git-bisect: Use binary search to find the commit that introduced a bug*. <https://git-scm.com/docs/git-bisect>.
- [38] GitLab. *GitLab CI/CD Documentation*. <https://docs.gitlab.com/ee/ci/>.